Università degli Studi di Palermo

dipartimento di ingegneria unipa

Introduzione a pandas

INTELLIGENZA ARTIFICIALE PER L'INGEGNERIA BIOMEDICA (EX ROBOTICA MEDICA)

a.a. 2025/2026

Prof. Roberto Pirrone

CHILAB

Creazione di un DataFrame

pandas — struttura dati tabellare con indice su righe e colonne

Da liste e tuple

Python

```python
import pandas as pd
import numpy as np

# Da lista di liste (righe)
df = pd.DataFrame([[1,'Alice',30],[2,'Bob',25]],
                  columns=['id','nome','età'])

# Da lista di tuple
data = [(1,'Alice',30),(2,'Bob',25)]
df = pd.DataFrame(data, columns=['id','nome','età'])
```

Da dizionario (colonne → valori)

```python
# Chiavi = nomi colonne, valori = liste
df = pd.DataFrame({
    'id':   [1, 2, 3],
    'nome': ['Alice', 'Bob', 'Carlo'],
    'età':  [30, 25, 35]
})

# Da lista di dizionari (righe)
df = pd.DataFrame([{'id':1,'nome':'Alice'},{'id':2,'nome':'Bob'}])
```

Da array NumPy

Python

```python
# Array 2D → DataFrame
arr = np.array([[1,30],[2,25],[3,35]])
df  = pd.DataFrame(arr, columns=['id','età'])

# Con indice personalizzato
df = pd.DataFrame(arr,
                  columns=['id','età'],
                  index=['a','b','c'])

# Da Series
s = pd.Series([10, 20, 30], name='valore')
df = s.to_frame()          # Series → DataFrame
```

Attributi principali del DataFrame

```python
df.shape           # (n_righe, n_colonne)
df.columns         # Index dei nomi colonna
df.index           # RangeIndex o Index personalizzato
df.dtypes          # dtype di ogni colonna
df.values          # array NumPy sottostante
df.size            # totale elementi (rows*cols)

# Convertire tipi
df['età'] = df['età'].astype(float)
df['data'] = pd.to_datetime(df['data_str'])
```

Import ed export del DataFrame

CSV · JSON · SQL — lettura e scrittura

CSV — lettura e scrittura

Python

```python
# Lettura
df = pd.read_csv('dati.csv')
df = pd.read_csv('dati.csv',
                 sep=';',           # separatore
                 header=0,          # riga intestazione
                 index_col='id',    # colonna come indice
                 usecols=['a','b'], # solo alcune colonne
                 na_values=['N/A','-'],
                 dtype={'età': int})
```

CSV — scrittura

```python
df.to_csv('output.csv', index=False)
df.to_csv('output.csv',
          sep=';',
          decimal=',',         # virgola decimale
          float_format='%.3f',
          encoding='utf-8',
          na_rep='NA')
```

JSON e altri formati

Python

```python
# JSON
df = pd.read_json('dati.json')
df.to_json('output.json', orient='records', indent=2)

# Excel
df = pd.read_excel('dati.xlsx', sheet_name='Foglio1')
df.to_excel('output.xlsx', sheet_name='Dati', index=False)

# Parquet (colonnare, efficiente)
df = pd.read_parquet('dati.parquet')
df.to_parquet('output.parquet', compression='snappy')
```

SQL — lettura e scrittura

```python
import sqlalchemy as sa

engine = sa.create_engine('sqlite:///db.sqlite')

# Lettura da query SQL
df = pd.read_sql('SELECT * FROM pazienti', engine)
df = pd.read_sql_query('SELECT id,nome FROM t', engine)
df = pd.read_sql_table('pazienti', engine)

# Scrittura su tabella SQL
df.to_sql('pazienti', engine,
          if_exists='replace', index=False)
```

Accesso ai dati

[] · notazione oggetto · loc · iloc · at · iat

Selezione di colonne: [] e notazione oggetto

Python

```python
# Colonna singola → Series
s = df['nome']
s = df.nome         # notazione oggetto (no spazi/keyword)

# Più colonne → DataFrame
sub = df[['nome', 'età']]

# Slice di righe (solo su RangeIndex)
df[2:5]             # righe 2,3,4
df[::-1]            # ordine inverso

# Colonna booleana
mask = df['età'] > 30   # Series booleana
df[mask]                # filtraggio
```

loc — selezione per etichetta

```python
# df.loc[row_label, col_label]
df.loc[0]                  # riga con label 0
df.loc[0, 'nome']          # cella singola
df.loc[[0,1,2], 'nome':'età']  # slice colonne
df.loc[df['età']>30, ['nome','età']]

# Assegnazione con loc
df.loc[df['età']>60, 'fascia'] = 'senior'
```

iloc — selezione per posizione intera

Python

```python
# df.iloc[row_pos, col_pos]
df.iloc[0]               # prima riga
df.iloc[-1]              # ultima riga
df.iloc[0, 2]            # riga 0, colonna 2

# Slice di righe e colonne
df.iloc[0:3]             # prime 3 righe
df.iloc[:, 1:3]          # colonne 1 e 2
df.iloc[0:4, 0:2]        # sottoinsieme rettangolare

# Liste di posizioni
df.iloc[[0, 2, 4]]       # righe 0, 2, 4
df.iloc[[0,1], [0,2]]    # righe 0-1, col 0 e 2
```

at e iat — accesso a cella singola (rapido)

```python
# at → per etichetta (come loc ma scalare)
val = df.at[3, 'nome']     # lettura
df.at[3, 'nome'] = 'Dora'  # scrittura

# iat → per posizione intera (come iloc scalare)
val = df.iat[3, 1]         # lettura
df.iat[3, 1] = 'Dora'      # scrittura
```

Assi del DataFrame e filtraggio booleano

axis=0 / axis=1 · operatori booleani · query()

Gli assi di un DataFrame

Asse	alias	Direzione	Esempio
axis=0	'index'	↕ lungo le righe	df.sum(axis=0) → per colonna
axis=1	'columns'	↔ lungo le colonne	df.sum(axis=1) → per riga

Trasposta, rename, set_index, reset_index

```
df.T                        # trasposta (righe ↔ colonne)

# Rinominare colonne
df = df.rename(columns={'nome':'name','età':'age'})
df.columns = ['id','name','age']  # sostituzione diretta

# Impostare/rimuovere l'indice
df = df.set_index('id')      # id diventa indice
df = df.reset_index()        # ritorna a RangeIndex
```

Filtraggio con espressioni booleane

Python

```
# Operatori di confronto: ==, !=, <, >, <=, >=
df[df['età'] > 30]
df[df['nome'] == 'Alice']

# Combinazione con & (and), | (or), ~ (not)
df[(df['età'] > 25) & (df['età'] < 40)]
df[(df['nome']=='Alice') | (df['nome']=='Bob')]
df[~(df['età'] > 60)]           # negazione

# isin e between
df[df['nome'].isin(['Alice','Bob','Carlo'])]
df[df['età'].between(20, 35)]

# str accessor (colonne testo)
df[df['nome'].str.startswith('A')]
df[df['nome'].str.contains('li', case=False)]

# query() — sintassi SQL-like
df.query('età > 25 and nome != "Bob"')
soglia = 30
df.query('età > @soglia')       # variabile locale con @
```

■ Booleane

■ Usare sempre parentesi intorno a ogni condizione con & e |.
query() è più leggibile ma non supporta tutti gli accessor pandas.

Prima analisi dei dati

head · tail · info · isna · duplicated

Ispezione iniziale del DataFrame

Python

```python
df.head(5)           # prime 5 righe (default)
df.tail(3)           # ultime 3 righe

# info(): struttura compatta
df.info()            # dtype, non-null count, memoria
df.info(memory_usage='deep')  # uso memoria reale

# Conteggio valori per colonna
df['genere'].value_counts()
df['genere'].value_counts(normalize=True)  # %

df.nunique()         # valori unici per colonna
df['col'].unique()   # array valori unici
```

Valori mancanti (NaN / None)

```python
# Individuazione
df.isna()            # DataFrame booleano
df.isnull()          # alias di isna()
df.isna().sum()      # conteggio NA per colonna
df.isna().sum(axis=1) # NA per riga
df.isna().any()      # True se almeno 1 NA
df.isna().mean() * 100  # percentuale NA
```

Individuazione di righe duplicate

Python

```python
# duplicated() → Series booleana
df.duplicated()      # True per ogni duplicato
df.duplicated(keep='first')  # mantieni prima occorrenza
df.duplicated(keep=False)    # marca TUTTI i duplicati

# Subset di colonne
df.duplicated(subset=['nome','età'])

# Contare e visualizzare duplicati
df.duplicated().sum()        # quanti duplicati
df[df.duplicated(keep=False)]# visualizza i duplicati
```

Analisi di base delle distribuzioni

```python
# describe() — statistiche colonne numeriche
df.describe()        # count, mean, std, min, quartili, max
df.describe(include='all')  # include anche categoriche
df.describe(percentiles=[.1,.25,.75,.9])

# Singole statistiche
df['età'].mean()     # media
df['età'].median()   # mediana
df['età'].std()      # deviazione standard
df['età'].skew()     # asimmetria (skewness)
```

Gestione NA e modifica della struttura

dropna · fillna · drop · insert — operazioni in-place vs copy

Rimozione valori mancanti — dropna()

Python

```python
# Rimuove righe con almeno un NA
df_clean = df.dropna()
df_clean = df.dropna(axis=0)        # lungo righe (default)
df_clean = df.dropna(axis=1)        # rimuove colonne con NA

# Controllo soglia
df.dropna(thresh=3)   # mantieni righe con ≥3 valori non-NA
df.dropna(subset=['nome','età'])  # solo in queste colonne
```

Riempimento valori mancanti — fillna()

```python
# Valore costante
df.fillna(0)                 # riempi tutti con 0
df['età'].fillna(df['età'].mean())  # con la media

# Propagazione
df.fillna(method='ffill')    # forward fill
df.fillna(method='bfill')    # backward fill
df.fillna(method='ffill', limit=2)  # max 2 propagazioni

df.interpolate()             # interpolazione lineare
```

Aggiunta e rimozione di colonne

Python

```python
# Aggiungere colonna (assegnazione diretta)
df['bmi'] = df['peso'] / df['altezza']**2
df['fascia'] = 'standard'     # valore costante

# insert() — posizione specifica
df.insert(2, 'bmi', df['peso']/df['altezza']**2)

# Rimuovere colonne
df = df.drop(columns=['id','extra'])
df = df.drop(columns='id')          # colonna singola
```

Aggiunta e rimozione di righe

```python
# Aggiungere righe — pd.concat (raccomandato)
nuova = pd.DataFrame([{'nome':'Elena','età':28}])
df    = pd.concat([df, nuova], ignore_index=True)

# Rimuovere righe per etichetta
df = df.drop(index=3)           # singola riga
df = df.drop(index=[1, 3, 5])   # più righe

# Rimuovere per condizione
df = df[df['età'] >= 18]        # filtraggio
df = df.drop(df[df['età']<18].index)
```

Analisi numerica e query booleane

describe · corr · where · any · all

describe() e correlazione

Python

```python
# Statistica descrittiva
df.describe()               # count,mean,std,min,q,max

# Singole statistiche (colonna o DataFrame intero)
df.mean(numeric_only=True)  # media colonne numeriche
df.std()
df.min(); df.max()
df.quantile([.25,.50,.75])
df.kurt()                   # curtosi
df.skew()                   # asimmetria
```

Matrice di correlazione — corr()

```python
# Pearson (default)
corr = df.corr(numeric_only=True)
corr['età']               # correlazioni con età

# Spearman (rank) e Kendall
df.corr(method='spearman')
df.corr(method='kendall')
```

where() e mask()

Python

```python
# where(): mantieni dove True, NaN altrove
df['età'].where(df['età'] >= 18)        # <18 → NaN
df['età'].where(df['età'] >= 18, other=0)  # <18 → 0

# mask(): opposto di where (sostituisce dove True)
df['età'].mask(df['età'] > 100, other=np.nan)

# Applicare su DataFrame intero
df.where(df > 0)            # negativi → NaN
df.mask(df < 0, other=0)    # negativi → 0
```

any() e all() — test aggregati

```python
# any(): True se almeno un elemento è True
df.isna().any()          # colonne con almeno 1 NA
df.isna().any(axis=1)    # righe con almeno 1 NA

# all(): True se tutti gli elementi sono True
df['età'].gt(0).all()    # tutte le età > 0?
(df > 0).all(axis=1)     # tutte le colonne > 0 per riga?

# Combinazione con filtraggio
df[df.gt(0).all(axis=1)] # righe con tutti valori > 0
df.dropna(how='any')     # rimuovi righe con qualsiasi NA
```

Raggruppamenti e trasformazioni

groupby · agg · apply · transform

groupby() — aggregazione per gruppi

Python

```python
# Raggruppamento base
g = df.groupby('reparto')

# Funzioni aggregate standard
g['stipendio'].mean()        # media per reparto
g['stipendio'].sum()
g.size()                     # dimensione di ogni gruppo
g['età'].agg(['mean','std','min','max'])

# Più colonne di raggruppamento
df.groupby(['reparto','genere'])['stipendio'].mean()
```

agg() — aggregazioni multiple

```python
# Dizionario colonna → funzione/lista
df.groupby('reparto').agg(
    stipendio_medio=('stipendio','mean'),
    n_dipendenti=('nome','count'),
    età_max=('età','max')
)

# Funzione personalizzata
df.groupby('reparto')['stipendio'].agg(
    lambda x: x.quantile(0.9))
```

apply() — funzioni arbitrarie su gruppi

Python

```python
# apply su Series (per elemento)
df['nome'] = df['nome'].apply(str.upper)
df['età']  = df['età'].apply(lambda x: x*2)

# apply su DataFrame (per colonna o riga)
df.apply(np.sum)             # somma per colonna (axis=0)
df.apply(np.sum, axis=1)     # somma per riga

# apply su GroupBy
def zscore(x):
    return (x - x.mean()) / x.std()
df.groupby('reparto')['stipendio'].apply(zscore)
```

transform() e pipe()

```python
# transform(): stessa dimensione del DF originale
df['mean_reparto'] = df.groupby('reparto')['stipendio']\
                       .transform('mean')

# pipe(): catena di operazioni leggibile
result = (df
    .pipe(pd.DataFrame.dropna)
    .pipe(pd.DataFrame.query, 'età > 18')
    .pipe(lambda d: d.assign(bmi=d.peso/d.altezza**2))
)
```

merge, pivot e plot

Unione di DataFrame · tabelle pivot · visualizzazione

merge() — join tra DataFrame

Python

```
# merge base (SQL-style join)
result = pd.merge(df_left, df_right, on='id')

# how: 'inner','left','right','outer','cross'
result = pd.merge(df1, df2,
                  left_on='id_paz',
                  right_on='paziente_id',
                  how='left')

# Chiavi multiple
result = pd.merge(df1, df2,
                  on=['anno','mese'],
                  suffixes=('_sx','_dx'))
```

concat() e join()

```
# Concatenazione verticale
df_tot = pd.concat([df1, df2, df3], ignore_index=True)
# Concatenazione orizzontale
df_tot = pd.concat([df1, df2], axis=1)
result = df1.join(df2, on='id', how='left')
```

pivot_table() — tabella pivot

Python

```
pt = pd.pivot_table(df,
    values='stipendio',
    index='reparto',
    columns='genere',
    aggfunc='mean',     # o 'sum','count',np.std…
    fill_value=0,       # sostituisce NaN
    margins=True)       # aggiunge riga/colonna totale

pt.reset_index()        # converte pivot in DataFrame
```

plot() — grafici integrati

```
import matplotlib.pyplot as plt

df['età'].plot(kind='hist', bins=20)
df.plot(x='età', y='stipendio', kind='scatter')
df['reparto'].value_counts().plot(kind='bar')
df[['stipendio','età']].plot(kind='box')
df['stipendio'].plot(kind='kde')   # densità

# Matrice di scatter
from pandas.plotting import scatter_matrix
scatter_matrix(df, diagonal='hist', figsize=(8,8))
```

Riepilogo e Risorse

pandas in una pagina

Argomenti trattati

- **Creazione:** da liste, tuple, dict, NumPy array, Series
- **Import / Export:** read_csv, read_json, read_sql, to_csv, to_excel, to_parquet
- **Accesso []:** colonne singole/multiple, slice di righe, maschera booleana
- **loc / iloc:** selezione per etichetta / posizione, slice, liste
- **at / iat:** accesso e scrittura su cella singola (più rapido di loc/iloc)
- **Assi:** axis=0 (righe) · axis=1 (colonne) — trasposta, rename, set_index
- **Filtraggio bool.:** ==,!=,<,> · & | ~ · isin · between · str.contains · query()
- **Prima analisi:** head, tail, info, value_counts, nunique, isna, duplicated
- **Gestione NA:** dropna(thresh,subset) · fillna(method,limit) · interpolate()
- **Righe / colonne:** drop(columns=…), drop(index=…), insert(), concat()
- **describe / corr:** statistiche descrittive, matrice di correlazione Pearson/Spearman
- **where / any / all:** maschere condizionali, test aggregati lungo assi
- **groupby / agg:** raggruppamento, agg() con nomi, funzioni personalizzate
- **apply / transform:** apply su Series, DataFrame, GroupBy — transform per shape uguale
- **merge / concat:** 'inner','left','right','outer' · suffixes · concat verticale/oriz.
- **pivot_table:** values, index, columns, aggfunc, fill_value, margins
- **plot():** hist, scatter, bar, box, kde — scatter_matrix

Risorse ufficiali

- **Documentazione pandas:** https://pandas.pydata.org/docs/
- **User Guide:** https://pandas.pydata.org/docs/user_guide/
- **API Reference:** https://pandas.pydata.org/docs/reference/
- **10 minutes to pandas:** https://pandas.pydata.org/docs/user_guide/10min.html
- **Cheat Sheet (ufficiale):** https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf
- **Cookbook (ricette avanzate):** https://pandas.pydata.org/docs/user_guide/cookbook.html
- **Visualizzazione integrata:** https://pandas.pydata.org/docs/user_guide/visualization.html

Ecosistema e librerie complementari

`numpy` — base computazionale (array, ufunc, linalg)

`matplotlib` — visualizzazioni avanzate — usato da pandas.plot()

`seaborn` — statistiche — heatmap corr, pairplot, boxplot

`scikit-learn` — ML — pipeline con ColumnTransformer e DataFrame

`polars` — alternativa pandas in Rust — molto più veloce su grandi dati

`dask` — pandas parallelo su dati che non entrano in RAM

`sqlalchemy` — ORM SQL per read_sql / to_sql su qualsiasi RDBMS